

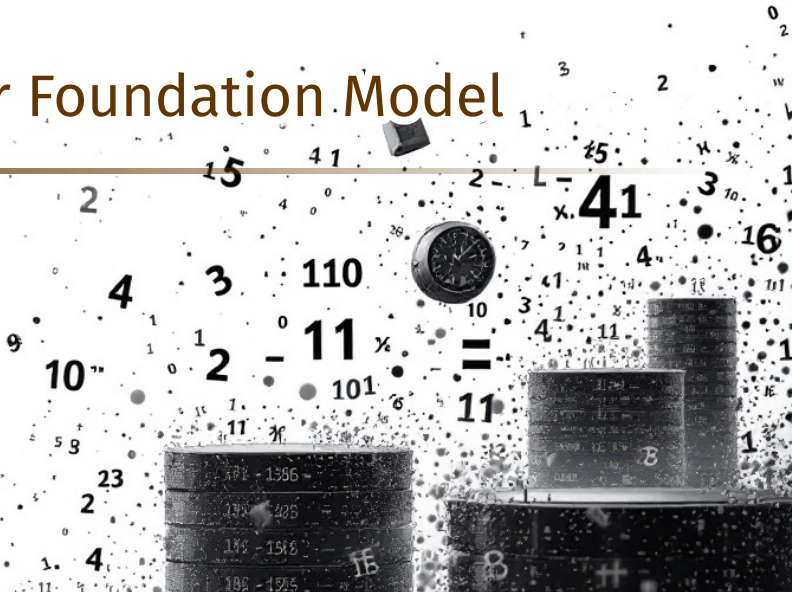
TabICL

Open Tabular Foundation Model

Gaël Varoquaux

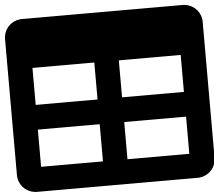
Inria

:Probabl.



1 Tabular learning

Tables are the thing



Tabular learning is everywhere

In Science

Tables of measures & observations

Throughout society

Predicting:

- Credit-card fraud
- Health risks
- Airplane delays
- Cyber attacks
- ...

Machine learning from tables
is important



‘AI’: amazing breakthroughs
on images, sound, text

But the most precious data is
in tables

Learning on tables
is not as easy as it seems

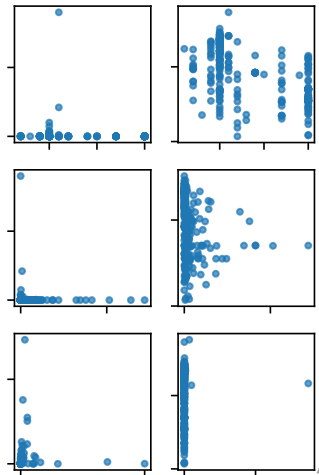


Tabular data: statistical properties

- Columns of different nature, different distributions
- Categorical data

Tree based models' inductive biases

Sex	Experience	Age
M	10 yrs	42
F	23 yrs	NA
M	3 yrs	28
F	16 yrs	45
M	13 yrs	48
M	6 yrs	36
M	NA	62



2022: tree models > deep learning

[Grinsztajn... 2022]

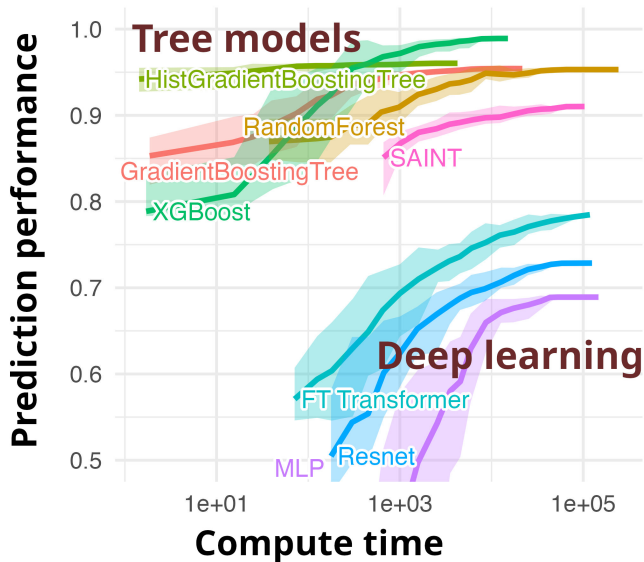
Tree-based methods
out-perform tailored
deep architectures



sklearn

HistGradientBoosting...

Faster **and** better predictor



Tabular data ... good for trees

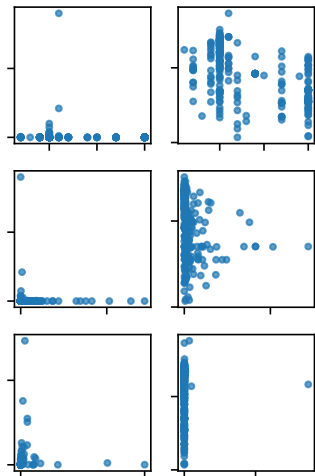
[Grinsztajn... 2022]

Little regularities

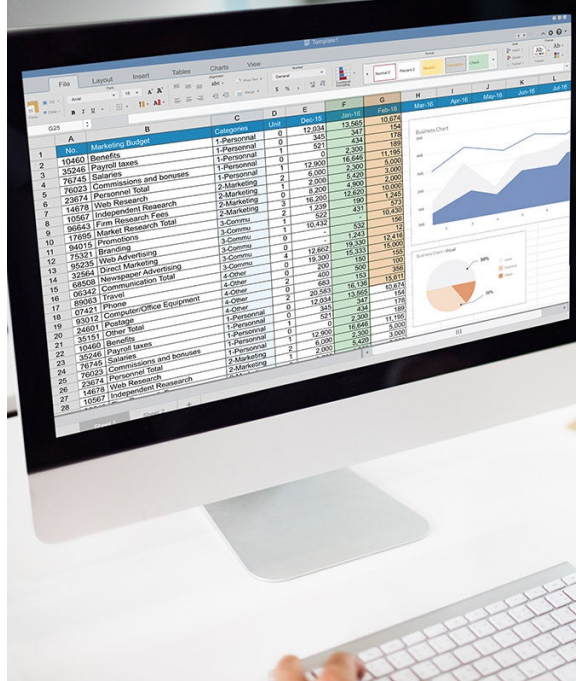
■ Columns of different distributions ... **trees don't care**

■ Categorical data ... **trees don't care**

Sex	Experience	Age
M	10 yrs	42
F	23 yrs	NA
M	3 yrs	28
F	16 yrs	45
M	13 yrs	48
M	6 yrs	36
M	NA	62



Neural Networks: smoothness inductive bias

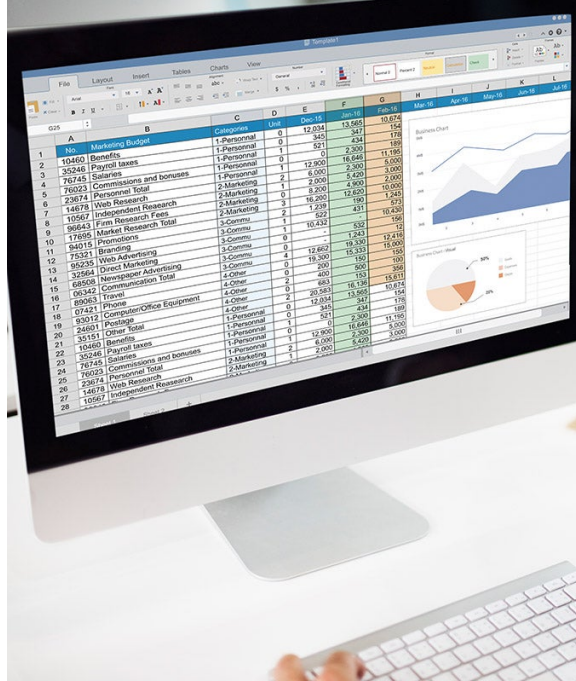


Neural Networks: smoothness inductive bias

Progress in neural networks
(capturing data regularities)
in vision, text...

- Large data
- Pretrained

Table Foundation Models?



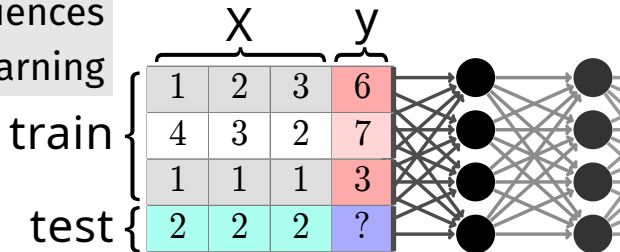
Transformers as programmable few-shots learners [Hollmann... 2023]

In context learning with transformers

Language models complete sequences
 $\Rightarrow$ few-shots learning

TabPFN

Apply this to tables



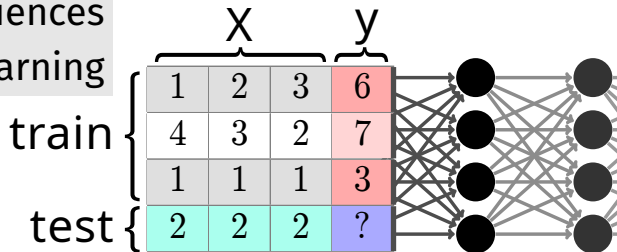
Transformers as programmable few-shots learners [Hollmann... 2023]

In context learning with transformers

Language models complete sequences
 $\Rightarrow$ few-shots learning

TabPFN

Apply this to tables



■ Teach transformers to complete tables (meta learning)

Many synthetic tables

Defines the regularities:

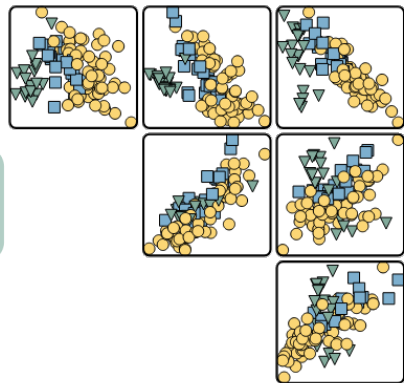
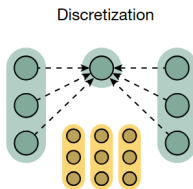
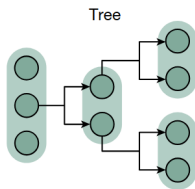
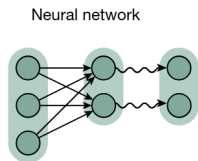
- Columns function of others
- Non-Gaussian marginals

1. Elaborate synthetic data

■ structural graph

■ trees

■ discretize



2. Flexible & adapted architecture

1. Elaborate synthetic data

2. Flexible & adapted architecture

Rows are composite objects

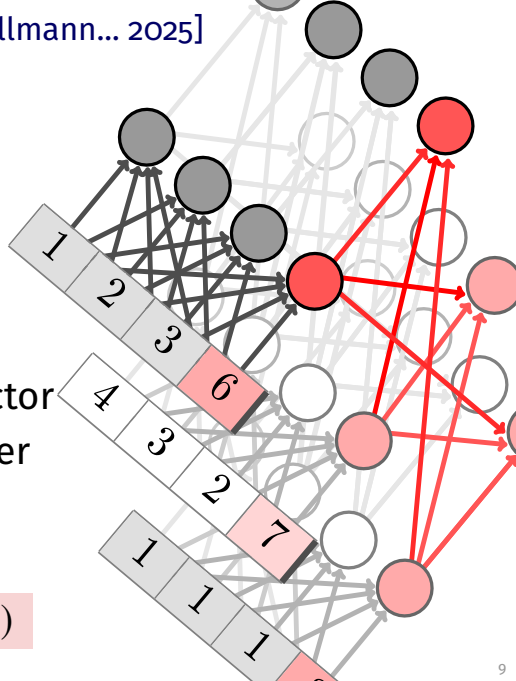
2D structure

■ col “position encoding”: random vector

■ Invariance to rows and columns order

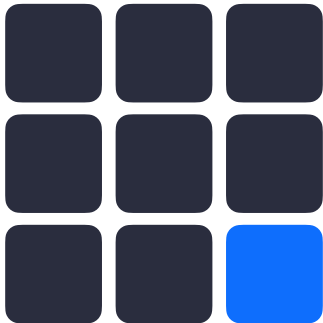
⇒ alternating row-wise
and col-wise attentions

Computational cost $O(p n^2 + n p^2)$



2 TabICL

Leading edge, open-source, and state-of-the-art



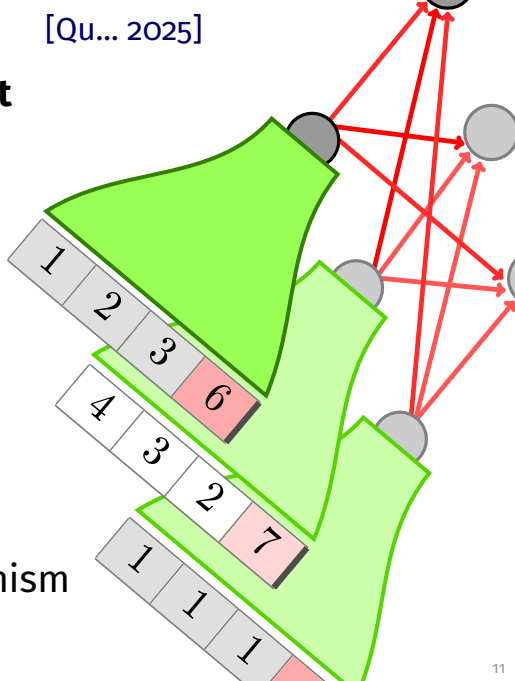
1. Break that quadratic cost

2-step architecture

1. Embed rows into fixed-dimensional representation
2. In context learning between these

■ $O(np + np^2 + n^2) = O(np^2 + n^2)$

- Needs to craft good encoding based on attention mechanism



2. Better pretraining

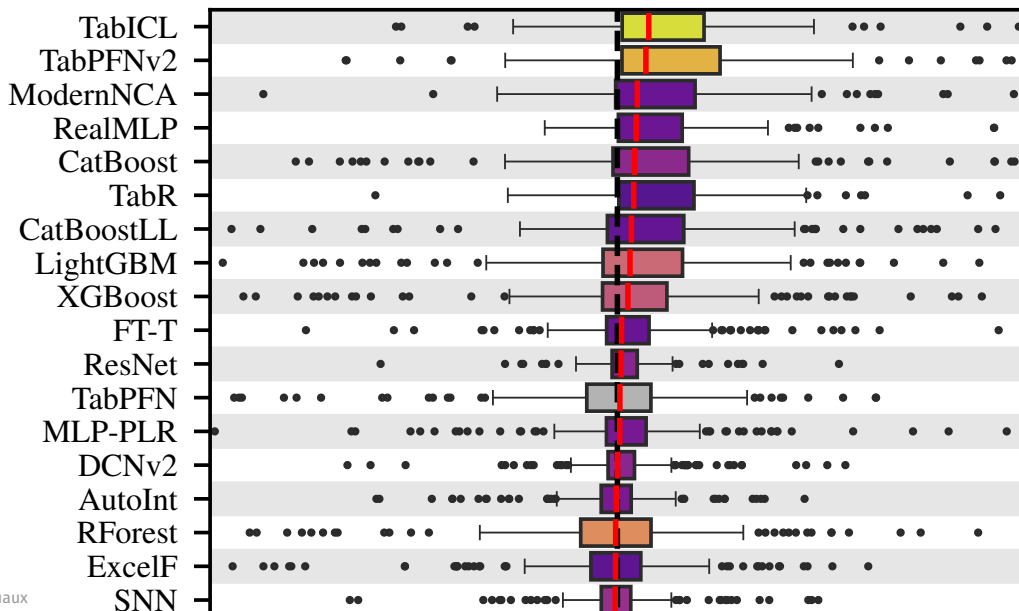
- Model is faster, more scalable
- Model adapts to marginal

Better synthetic data

- Bigger datasets
- More classes
- Richer data-generating non linearities

TabICL for the win 28 models, TALENT benchmark

[Qu... 2025]



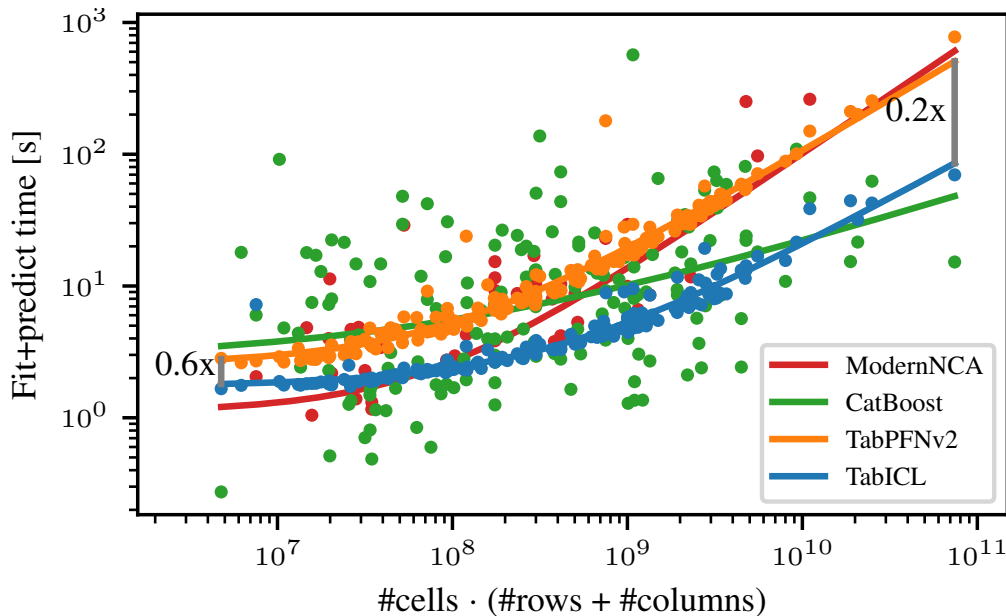


Table Foundation Models: Raging competition

TabPFN

TabICL

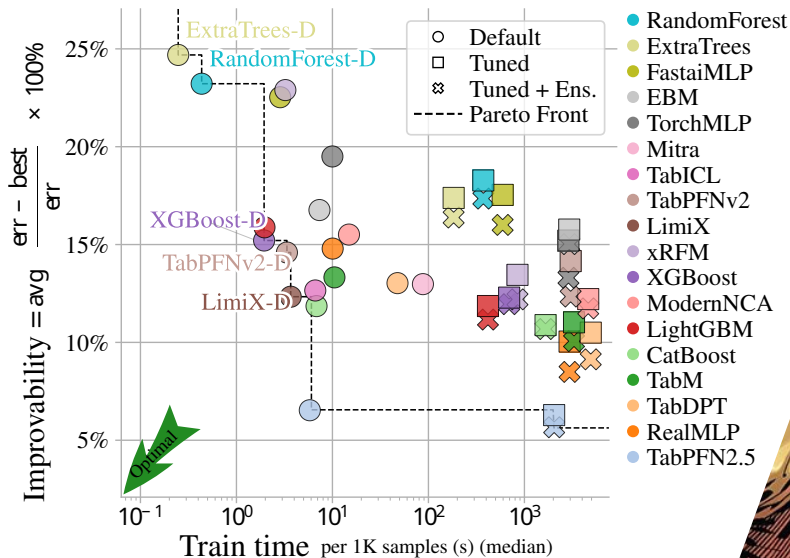
TabDPT

Mitra

The Arena

tabarena

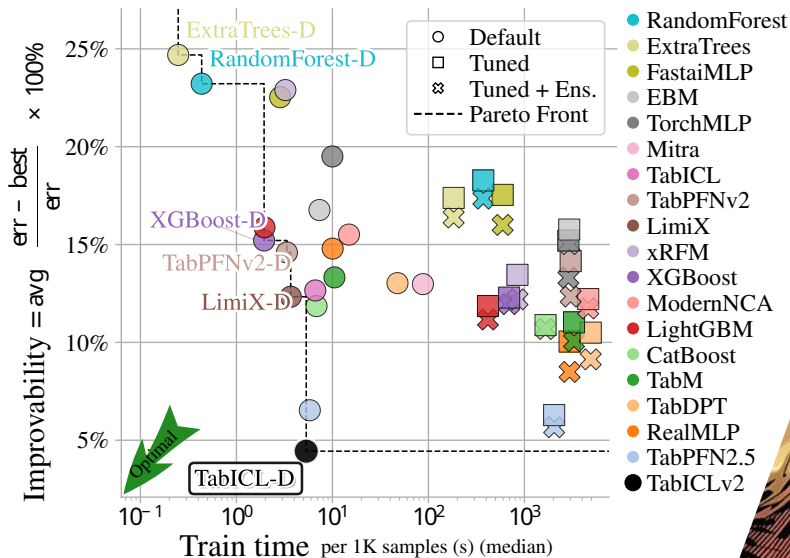
[Erickson... 2025]



The Arena

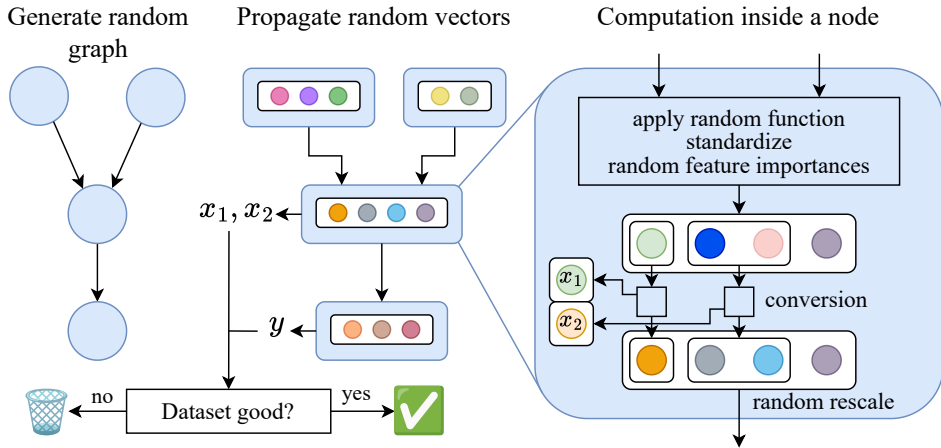
tabarena

[Erickson... 2025]



TabICLv2 on Arxiv

TabCLv2 Pretraining data generation



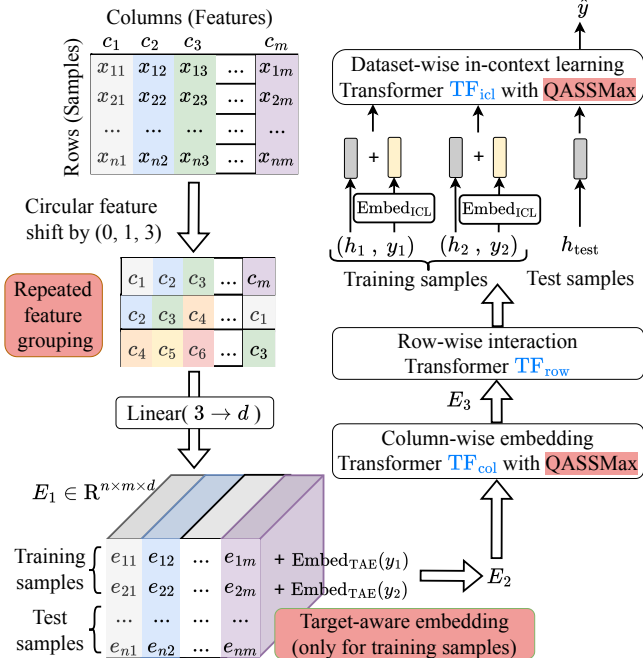
- Dataset rejection for curriculum learning
- Very rich function class on compute graph

S. Varoquaux

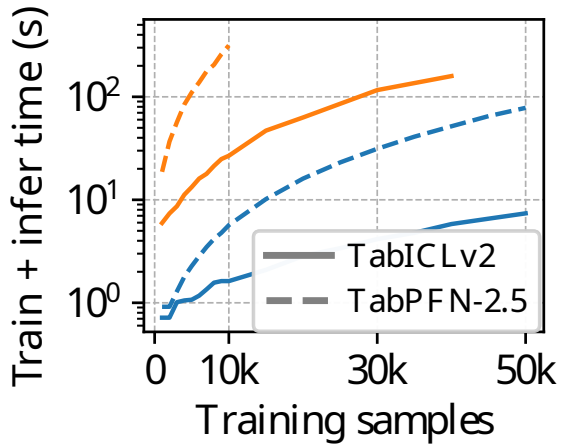


TabICLv2 Architecture

- Feature grouping
- Target-aware positional embedding
- Query-aware scalable softmax
trainable temperature scaling
(function of n)
Large tables need focus on
smaller fraction of samples

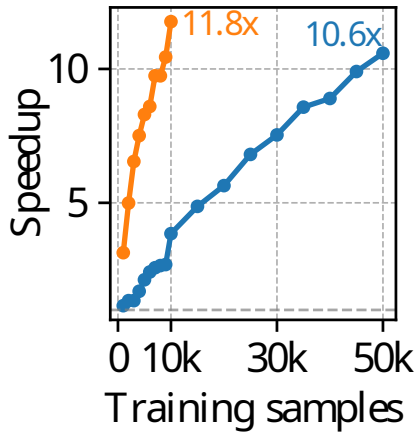


TabICLv2 computational cost



— NVIDIA GPU
(H100 NVL, 94Gb)

TabICLv2 vs. TabPFN-2.5

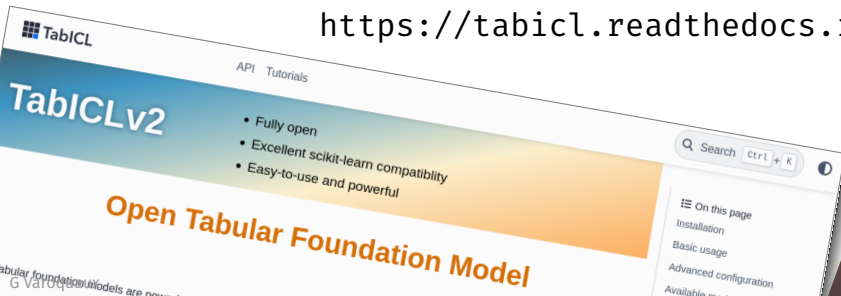


— AMD CPU
(EPYC, 48 cores)

Tabular-learning rebooted

- Tree-based methods have fallen
- Pretrained-transformers for in-context learning
 - Devil in the details
 - and in the quadratic attention cost
- TabICL: open-source, quality code

<https://tabicl.readthedocs.io>



```
from tabicl import TabICLClassifier
clf = TabICLClassifier( )
# checkpoint downloaded once on first run
clf.fit(X_train, y_train)
# in-context learning happens here
clf.predict(X_test)
```

TabICL: the software

<https://tabicl.readthedocs.io/>

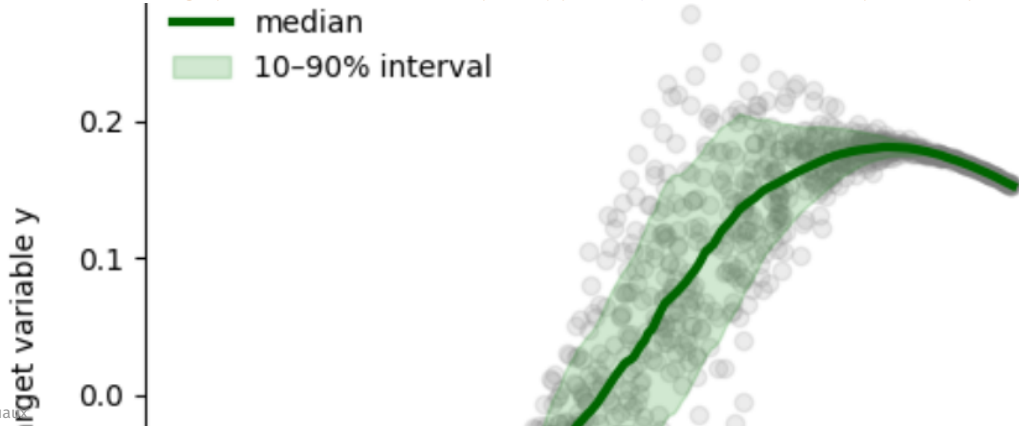
Feature complete: **Prediction quantiles**

```
reg = TabICLRegressor()
```

```
reg.fit(X_train, y_train)
```

```
alphas = [0.10, 0.5, 0.90]
```

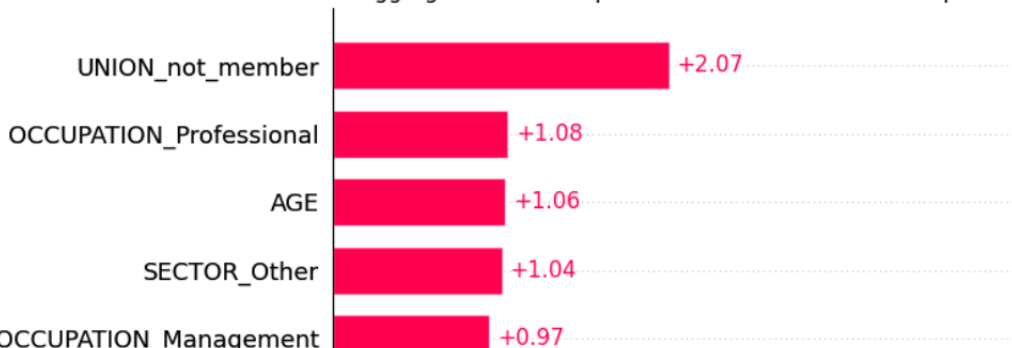
```
quantiles = reg.predict(X_test, output_type="quantiles", alphas=alphas)
```



Feature complete: **Fast Shap-like interpretability**

```
from tabicl.shap import get_shap_values, plot_shap
sv = get_shap_values(clf, X_test[:10])
plot_shap(sv, kind="bar")
```

Aggregate feature importances across the test examples

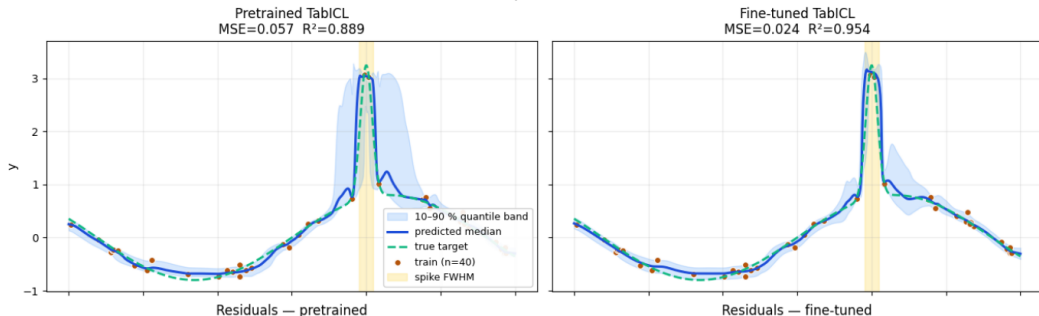


Feature complete: **Fine-tuning on your data**

```
reg = FinetunedTabICLRegressor(  
    epochs=60, learning_rate=1e-5, early_stopping=True,  
)
```

```
reg.fit(X_train, y_train, X_val=X_val, y_val=y_val)
```

Predictions + residuals: pretrained vs. fine-tuned

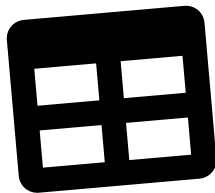


Feature complete

Open source
From *Inria* and **:Probabl.**
with ❤️

3 Skrub: heterogeneous dataframes

Tables are the thing





Big Data Borat

@BigDataBorat



In Data Science, 80% of time spent prepare data, 20% of time spent complain about need for prepare data.

Data munging is killing us

It's mostly pandas-level data preparation

Transforming tables for machine learning

- Default heuristics for different columns:

```
tab_vec = skrub.TableVectorizer()
```

```
X = tab_vec.fit_transform(df)
```

```
learner = sklearn.pipeline.make_pipeline(  
    skrub.TableVectorizer(),  
    tabicl.TabICLClassifier()  
)
```

Strong baseline



```

9 from sklearn.ensemble import HistGradientBoostingRegressor
14 from sklearn.metrics import mean_absolute_error, r2_score
15
16 # — 1. Load data —
17 df = pd.read_csv("employees_salaries.csv")
18 print(f"Shape: {df.shape}")
19 print(df.head())
20 print("\nColumn dtypes:\n", df.dtypes)
21 print("\nMissing values:\n", df.isnull().sum())
22
23 # — 2. Split features / target —
24 target_col = "salary"
25 X = df.drop(columns=[target_col])
26 y = df[target_col]
27
28 # — 3. Identify column types —
29 categorical_cols = X.select_dtypes(include=["object", "category"]).columns
30 numeric_cols = X.select_dtypes(include="number").columns.tolist()
31
32 print(f"\nNumeric features ({len(numeric_cols)}): {numeric_cols}")
33 print(f"Categorical features ({len(categorical_cols)}): {categorical_cols}")
34
35 # — 4. Preprocessing —
36 # HistGradientBoosting natively handles NaNs in numeric columns.
37 # Categorical columns are ordinarily encoded (the model handles them as integers)
38 preprocessor = ColumnTransformer(
39     transformers=[
40         ("num", "passthrough", numeric_cols),
41         (
42             "cat",
43             OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1),
44             categorical_cols,
45         ),
46     ],
47     remainder="drop",
48 )
49
50 # — 5. Model pipeline —
51 pipeline = Pipeline(
52     steps=[
53         ("preprocessor", preprocessor),
54         (
55             "model",
56             HistGradientBoostingRegressor(
57                 max_iter=300,
58                 learning_rate=0.05,
59                 max_depth=6,
60                 random_state=42,
61             )
62         )
63     ]
64 )

```

```

9 from sklearn.ensemble import HistGradientBoostingRegressor
10 from sklearn.metrics import mean_absolute_error, r2_score
11+ from sklearn.model_selection import cross_val_score, train_test_split
12+ from sklearn.pipeline import make_pipeline
13
14 # — 1. Load data —
15 df = pd.read_csv("employees_salaries.csv")
16 print(f"Shape: {df.shape}")
17 print(df.head())
18
19 # — 2. Split features / target —
20+ X = df.drop(columns=["salary"])
21+ y = df["salary"]
22+
23+ # — 3. Pipeline —
24+ # TableVectorizer auto-detects column types and applies appropriate encoders.
25+ pipeline = make_pipeline(
26+     TableVectorizer(),
27+     HistGradientBoostingRegressor(
28+         max_iter=300, learning_rate=0.05, max_depth=6, random_state=42
29+     )
30+ )

```

The script is at predict_salary.py

Pipeline

1. Loads `employees_salaries.csv`
2. Auto-detects numeric vs. categorical columns
3. Preprocessing via `ColumnTransformer`
 - Numeric columns → `passthrough`
 - Categorical columns → `OrdinalEncoder`
4. Model: `HistGradientBoostingRegressor`
5. Evaluation: MAE and R^2 on cross-validation
6. Feature importances printed

Run it:

```
python predict_salary.py
```

If the `salary` column has a different name, adjust the script.

predict_salary.py

Please use skrub instead of the ColumnTransformer

Thought for 2s

Read predict_salary.py

Edit predict_salary.py

Removed 40 lines

```

"""
Salary prediction model
Uses a HistGradientBoostingRegressor
of numeric and categorical features
Uses skrub's TableVectorizer
"""

import numpy as np
import pandas as pd
import numpy as np
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

```

Make this edit to predict_salary.py

1 Yes

2 Yes, allow all edits this session

3 No

Better data-science primitives

Crucial in an era of agents

TabICL

Table Foundation Model

■ Powerful prediction without tuning

Skrub

Uniting ML & dataframes



- 1 Tabular learning
- 2 TabICL
- 3 Skrub: heterogeneous dataframes

Big picture and integration
through the data science pipeline



References I

- N. Erickson, L. Purucker, A. Tschalzev, D. Holzmüller, P. M. Desai, D. Salinas, and F. Hutter. TabArena: A living benchmark for machine learning on tabular data. In *Neural Information Processing Systems*, 2025.
- L. Grinsztajn, E. Oyallon, and G. Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *NeurIPS*, 2022.
- N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. *ICLR*, 2023.
- N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025.
- J. Qu, D. Holzmüller, G. Varoquaux, and M. Le Morvan. Tabicl: A tabular foundation model for in-context learning on large data. *arXiv:2502.05564*, 2025.
- J. Qu, D. Holzmüller, G. Varoquaux, and M. L. Morvan. Tabiclv2: A better, faster, scalable, and open tabular foundation model. *arXiv preprint arXiv:2602.11139*, 2026.